Proto-Hive Functionality Inventory: ZeroBot/OpenClaw and ProtoMegaBot/OmegaClaw

ZeroBot / ProtoCosmoBot operational inventory for OmegaHive implementers

Snapshot: 2026-07-07 21:05 Pacific (revision 2)

1 Purpose and scope

This document is an ASCII LaTeX inventory of changes made after the initial local installation of the proto-hive agents on the Pop!_OS OpenClaw laptop. It is intended as an overview for OmegaHive implementers: what functions, safety mechanisms, recurring jobs, integrations, project records, and code changes have proven useful enough that they should be considered for OmegaHive.

The two operational targets are:

1. **ZeroBot / ProtoCosmoBot**: the OpenClaw research prototyping assistant running in the main OpenClaw workspace.
2. **ProtoMegaBot / ProtomegaTron**: the OmegaClaw/PeTTa based bot, run locally on the same Pop!_OS laptop and routed through OpenClaw for LLM calls.

This is a best-effort operational inventory, not a forensic byte-level diff of the whole machine. It was compiled from current OpenClaw status/configuration, the cron registry, project records, memory logs, Git heads, and recovery manifests. Secrets, tokens, and credential values are intentionally omitted.

2 High-level themes for OmegaHive

The proto-hive has converged on several useful patterns:

1. **Persistent project notebooks**: every nontrivial project has PROJECT.md, TASKS.md, DECISIONS.md, NOTES.md, repositories, experiments, and artifacts under `projects/<slug>/`.
2. **Scheduled autonomous workers**: cron jobs drive project progress, recovery backups, bot-bot discussions, watchdog checks, scheduler reconciliation, and daily summaries.
3. **Dedicated communication channels**: direct chat for Ben, a main Protobots channel, a scheduled-updates channel, and a BotBotChats channel for bot-bot coordination.
4. **Safety-first recoverability**: private GitHub recovery repositories snapshot sanitized ZeroBot and ProtoMegaBot state daily.
5. **Model routing and fallback**: OpenClaw primary model is GPT-5.5 with GLM fallback; a cost/intent router sends routine work to cheaper models and substantive OmegaClaw prompts to GPT-5.5.

6. **Runtime watchdogs**: a local watchdog restarts ProtoMegaBot when it dies; a channel watchdog scans for interaction failures.

7. **Native subagent hardening**: ThreadKeeper/OmegaClaw subagents have been hardened with bounded transcripts, bounded tool outputs, queue audit fields, type checks, and safer shell behavior.

8. **Non-live gates before live autonomy**: GoalChainer, petta-memory evidence, and OmegaClaw/ZeroBot bridge work are exercised through non-live artifacts and GGB-style gates before changing live Telegram/runtime behavior.

9. **Installation/functionality knowledge base**: a durable lookup KB at `catalog/INSTALLATION_KB.md` plus structured JSONL entries at `catalog/install-kb/components.jsonl` and a search helper `bin/install-kb-search` so the agent consults installed components before making claims about availability.

10. **Local LLM fallback**: Ollama with `qwen2.5:7b` is installed, tested, and configured as a third-level fallback (GPT-5.5 → GLM-5.2 → local Qwen 2.5 7B); a keepalive cron restarts it if down. Pending Gateway restart to activate in the running agent.

11. **OmegaClaw Telegram context-overflow hardening**: `lib_llm_ext.py` context-overflow recovery with preemptive escalation, GPT-5.5-class escalation knob, and chunk-summary fallback; Telegram pending-message FIFO to avoid burst message concatenation/loss through a single buffer.

12. **OmegaClaw late-extension registry**: `lib_extensions.metta` / `src/extensions.py` status hook for `OMEGACLAW_LATE_EXTENSIONS`; `(extension-status)` reports loaded/failed/deferred/pending. Heavy deontic/directive imports kept deferred until a bounded/lazy loader or Prolog-backed API route is available.

13. **OmegaClaw Telegram routing fixes**: source-chat binding for dequeued inbound messages patched to prevent wrong-channel substantive replies; duplicate `COMMAND_RETURN` feedback compacted before storing `LAST_SKILL_USE_RESULTS` to prevent stale skill-result replay, continuation loops, and prompt ballooning.

14. **Daily health-check system**: a two-part daily health monitor — a health checker cron (08:30 Pacific) checking ProtoMegaBot supervisor, OpenClaw Gateway, Kanban freshness, backup recency, uncommitted work age, and log freshness; plus an alarm cron (10:00 Pacific) that alerts Ben if the health report file is missing.

3 Current ZeroBot / OpenClaw operational state

3.1 Runtime and model state

Observed current OpenClaw status:

- OpenClaw version: `2026.6.10`.
- Gateway uptime at snapshot: about 19h 38m; system uptime about 11d 2h.
- Current model: `openai/gpt-5.5` using OpenAI OAuth.

- Configured fallback: `openrouter/z-ai/glm-5.2`.
- Gateway mode: local, loopback bind, port `18789`; authenticated OpenAI-compatible `/v1/chat/completions` and `/v1/responses` endpoints are enabled.
- Telegram channel is enabled; rich Telegram messages are currently off; standard Telegram formatting is available.
- The main workspace is `/home/openclaw/research-agent`.

3.2 Configuration changes from initial installation

Current notable OpenClaw configuration/functionality:

- Main assistant identity and context files: `AGENTS.md`, `SOUL.md`, `IDENTITY.md`, `USER.md`, `TOOLS.md`, `MEMORY.md`, and `HEARTBEAT.md`.
- Gateway HTTP endpoints enabled for local OpenAI-compatible access by OmegaClaw and smokes.
- Telegram direct and group routing configured for Ben and Protobots channels.
- Group-specific prompts configured for the main Protobots group, scheduled-updates channel, and ProtoBots-BotBotChats channel.
- Owner/user allowlist includes Ben's Telegram id for direct access and commands.
- Plugin loading enabled for the local `intent-model-router` plugin.
- Model defaults changed to GPT-5.5 primary with GLM fallback.
- Execution profile configured for coding/research work with broad local tool access on the dedicated laptop.

3.3 Installed or created skills

Bundled skills enabled in the assistant config include:

```
github, coding-agent, healthcheck, session-logs, tmux, summarize,
skill-creator, research-projects, experiment-ledger,
repository-operations, remote-compute-guardrails,
hyperon-workbench, knowledge-curation, research-library
```

Local research-agent skills currently present include:

```
cross-agent-kanban
daily-bot-bot-project-discussions
experiment-ledger
hyperon-workbench
kanban-task-board
knowledge-curation
plain-spec-governor
recurring-project-progress-worker
```

```
remote-compute-guardrails
repository-operations
research-library
research-projects
research-rules-checklist
```

Skill Workshop proposal status at snapshot:

- Applied: `cross-agent-kanban`, `kanban-task-board`, `research-rules-checklist`, `daily-bot-bot-projec`
  `recurring-project-progress-worker`, `plain-spec-governor`.
- Pending: `materialize-narrowing-gate-procedure`.
- Rejected stale/duplicate proposals include earlier variants of `plain-spec-governor`, `recurring-project-pr`
  and `specatom-review-check-scaffold`.

3.4 ZeroBot code and repository additions

Important local additions and reusable components:

- `plugins/intent-model-router`: local OpenClaw plugin for intent/cost/model routing.
- `projects/openclaw-intent-model-router/repos/openclaw-intent-model-router`: standalone public reusable repository extracted from the local plugin.
- `projects/agent-recovery`: private recovery project for ZeroBot and ProtoMegaBot.
- `projects/channel-watchdog`: channel watchdog project for detecting dropped continuations and channel interaction bugs.
- `catalog/KANBAN.md` and related cross-project catalog mechanisms for active task visibility.
- `setup-kit/`: reusable setup/config examples for OpenClaw installation and migration.

Current repository heads relevant to self/agent infrastructure:

```
projects/agent-recovery/repos/zerobot-recovery
  main 10fd8a5 Update ZeroBot recovery snapshot
projects/agent-recovery/repos/protomegabot-recovery
  main 25c8b3c Update Protomegabot recovery snapshot
projects/openclaw-intent-model-router/repos/openclaw-intent-model-router
  main a6c7171 Extract cost-aware OpenClaw intent router
```

3.5 Model routing

The `intent-model-router` began as a fix for ProtoMegaBot routing problems and was generalized into a reusable OpenClaw plugin. Operational behavior:

- Routine/simple work may route to `openrouter/z-ai/glm-5.2`.
- Substantive/deep work routes to `openai/gpt-5.5`.
- OmegaClaw/ProtomegaTron sessions are detected specially: simple chitchat can use GLM, while complex PeTTa/OmegaClaw prompts use GPT-5.5.
- The extracted repo includes tests, README/usage/security docs, and plugin metadata.

3.6 Local LLM fallback status

Ben requested a local LLM fallback so the bots do not become totally unresponsive during billing/provider failures. Current observed state after 2026-07-07 repair:

- The `ollama` binary exists at `/home/openclaw/.local/bin/ollama`, version 0.31.1.
- The `llama-server` binary and shared libraries were missing; they were installed from the official Ollama v0.31.1 Linux amd64 release tarball into `/home/openclaw/.local/lib/ollama/`.
- `ollama serve` is running and responding at `http://127.0.0.1:11434`.
- Model `qwen2.5:7b` (4.7 GB) pulled and smoke-tested ("2 + 2 equals 4").
- OpenClaw config updated: added provider `ollama` with `baseUrl: http://127.0.0.1:11434` and model `ollama/qwen2.5:7b`; appended after GLM in fallback list.
- An "Ollama keepalive" cron job (every 5 min, job id `b61b7f9a`) checks the API and restarts `ollama serve` if down.
- Pending: restart OpenClaw Gateway service (`sudo systemctl restart openclaw-agent.service`) for the running agent to pick up the new Ollama provider config.
- After restart, the fallback chain will be: GPT-5.5 → GLM-5.2 → local Qwen 2.5 7B.

Conclusion: local LLM fallback is now functionally installed and tested; it will become active in the running agent after the next Gateway restart.

3.7 Installation/functionality knowledge base

A durable install/functionality KB was created on 2026-07-07 after Ben noted the agent should not forget what has been installed:

- `catalog/INSTALLATION_KB.md`: human-readable Markdown KB with per-component entries covering status, version, path, role, and important notes.
- `catalog/install-kb/components.jsonl`: structured JSONL entries for programmatic lookup.
- `bin/install-kb-search <query>`: search helper that checks both the JSONL and Markdown sources.
- Update rule: whenever software, a runtime service, a toolchain, a plugin, a skill, a cron job, or a major local patch is installed/changed, add or update an entry here.
- The KB records: OpenClaw runtime, Gateway HTTP, Telegram integration, Tectonic LaTeX compiler, Ollama local LLM, Ollama keepalive cron, intent-model-router, agent recovery repos, channel watchdog, OmegaClaw runtime, OmegaClaw watchdog, local SWI-Prolog, PeTTa/OmegaClaw/ChromaDB stack, ThreadKeeper, GoalChainer, QMD, and core CLI tools.

OmegaHive recommendation: every hive agent should maintain a similar durable install/functionality KB with a search helper, and update it whenever components are added or changed.

4 Current cron jobs

The OpenClaw cron registry contained 39 jobs at this revision. The earlier table (38 jobs at 15:56) is unchanged except for one new job added since:

ID	Name	Enabled	Kind
70cd9d3f-8ed4-4d86-8887-917eb919c6b9	Channel watchdog scan	yes	every
9df4641e-f89a-4dca-9ed3-d749cf6f075c	omegaclaw-watchdog	yes	every
5a10c8f4-6b59-4069-9b77-7f2676086a79	botbot daily discussion: thread-keeper	yes	cron
f0d70a09-ab9f-40b9-ae3d-b7f789e375bf	ThreadKeeper hardening worker	yes	cron
f3c7b795-ec7d-4c9f-b9e6-090d00ab960e	mac-cla-orchestrator	yes	every
15d5d56c-d393-4916-9908-ceaab07b6a2d	Protobots GGB roadmap research worker	yes	cron
c008e434-ea10-4d75-b13b-26b922d079ed	petta-chem dedicated progress worker	yes	cron
4f4e146a-bdf9-4a6e-97da-6484cfe3f81f	petta-memory progress worker	yes	cron
dc7477b6-c991-409e-8ba7-d905767190e5	botbot daily discussion: ggb-roadmap	yes	cron
31e85b3b-784f-4b80-ab24-43e7167561b8	plain-to-metta progress worker	yes	cron
d8340b7b-16d8-4b4f-a758-d490b2ebe0c9	mac-cla-bihourly-report	yes	every
ff41614c-6484-452d-bc38-b469152923b8	botbot daily discussion: chaos-language-algorithm	yes	cron
1ad1f253-b010-4a1f-ba2f-40d0c90f4ac9	Daily OpenClaw skill/tool reflection	yes	cron
8e76228b-9be8-4d2d-be55-a3a5e9ba2a5a	Daily Kanban refresh	yes	cron
f994a3ea-9fb8-4bb3-b4cb-4e091ea3de12	Daily scheduler reconciliation (non-isolated)	yes	cron
5ae59dd5-dfe3-4ace-999d-a08ca153325e	Daily agent recovery GitHub backup	yes	cron
ede5677b-1688-4cdb-a265-f1253666118b	botbot discussion auto-discovery	yes	cron
d96e1fe8-021d-4278-bb4c-8429d69e516f	daily-subagent-summary	yes	cron
4807bf6e-b8a2-4775-aedf-e533b9de11ff	daily bot-bot project discussion - petta-chem	yes	cron
7e673013-71cf-4a67-9b8f-9c5523f7d345	Daily ecosystem health check	yes	cron
20031497-f585-43ad-91f9-d879d122d4e3	daily bot-bot project discussion - petta-memory	yes	cron

ab6eb308-bf3d-4fb0-8278-341b2e612e9a	botbot daily discussion: petta-memory	yes	cron
0ad8aee2-a1a2-4a9c-9bf7-f1d6413c13a9	botbot daily discussion: omegaclaw	yes	cron
b2b7d564-e93c-498b-84c9-70492e609e76	Daily health-check alarm	yes	cron
2587a873-c05e-4212-b0d6-beef473624bc	daily bot-bot project discussion - specatom-hs	yes	cron
c38e11d4-4d30-4789-8b02-6c3c38762966	botbot daily discussion: specatom-hs	yes	cron
51007476-bd4b-46ec-af74-de386880cd36	botbot daily discussion: hyperseed-formalizations	yes	cron
668dbe8d-96c4-4394-8388-5ab1f0404cff	daily bot-bot project discussion - hyperseed-formalizations	yes	cron
4d2a5788-01c9-4c09-a67e-b3223a7a0707	daily bot-bot project discussion - omegaclaw	yes	cron
a26edd76-0145-4c36-b722-8e481b88bb9f	daily bot-bot project discussion - agent-recovery	yes	cron
2fae9ef5-0c1a-49ec-b858-04d04ce289a7	botbot daily discussion: agent-recovery	yes	cron
62cf45f6-a858-4d10-aa05-ef632564529e	botbot daily discussion: relaleap	no	cron
6942a233-f2e0-4e9b-aec6-e6dbfe32c2a5	Check ProtomegaTron after ZeroBot test ping	no	at
6f8e458b-a288-419a-86b4-6006f167c329	daily bot-bot project discussion - omegasim	no	cron
6fb7b7bc-d541-4597-9dce-d0a4dad9cae4	petta-chem daily 7AM Pacific overview	no	cron
90c03ba2-3d96-4c00-99c0-105d6019430c	botbot daily discussion: omegasim	no	cron
9daaf2a7-1b58-4256-ae1c-dbe2c20c9b1c	Daily Protobots subagent progress summary	no	cron
a5d8122d-e026-4e82-b113-d9be41f1e12f	petta-chem two-hour brief progress update	no	every
hline b61b7f9a-4062-4d60-addc-71898e9ec239	Ollama keepalive	yes	every

Representative job details:

- **omegaclaw-watchdog**: every 10 minutes, isolated GLM agent, runs `bash /home/openclaw/research-ager` if ProtoMegaBot is down, restarts it and announces to Ben.
- **Channel watchdog scan**: every 15 minutes, scans visible Telegram bot conversations for dropped continuations, unfulfilled attachment promises, stock acknowledgements, runaway bot-bot chatter, and unsurfaced subagent completions; alerts scheduled-updates.
- **Daily agent recovery GitHub backup**: daily 03:30 Vancouver, pushes sanitized private recovery snapshots for ZeroBot and ProtoMegaBot; success is silent, failures alert scheduled-updates.

- **Daily scheduler reconciliation**: daily 00:45 Vancouver from a non-isolated maintenance session, because isolated cron jobs cannot manage other cron jobs. Reconciles daily bot-bot project discussion jobs and reports ambiguity/errors.
- **botbot discussion auto-discovery**: daily 06:30 Los Angeles, main-session system event that ensures active/idea projects have daily BotBotChats discussion jobs and disables paused/completed ones.
- **Daily OpenClaw skill/tool reflection**: daily 23:30 Vancouver, reviews repeated friction and proposes skills/tools/recurring tasks via Skill Workshop when useful.
- **Daily health checker**: daily 08:30 Pacific, runs `bin/health-check.sh` which checks ProtoMegaBot supervisor, OpenClaw Gateway, Kanban freshness, backup recency, uncommitted work age, and log freshness; writes `health-reports/YYYY-MM-DD.md`.
- **Daily health-check alarm**: daily 10:00 Pacific, alerts Ben if today's health report file is missing.
- **Ollama keepalive**: every 5 minutes, checks Ollama API and restarts if down.

5 Subagents and delegation

5.1 OpenClaw subagents

At this snapshot, no active or recent subagents were visible in the current main-agent session tree. Historically, the proto-hive uses OpenClaw subagents for isolated implementation/review slices, especially when a project needs parallel code work or audit work. Example recorded uses include:

- SpecAtom-HS Phase 2 semantic-object extraction subagent.
- RelaLeap SLT estimator branch/subagent waves.
- Project-specific workers triggered through cron jobs, which run isolated agent turns rather than persistent visible subagents.

5.2 OmegaClaw / ThreadKeeper subagents

ThreadKeeper is the main OmegaClaw-side subagent/delegation system. It has been hardened substantially, as described in Section 7. OmegaHive should treat this as evidence that subagents need native audit trails, transcript bounds, queued task contracts, typed final emits, type-checked tool calls, bounded tool outputs, and explicit adjudication of worker outputs.

6 ProtoMegaBot / OmegaClaw current state

6.1 Initial local OmegaClaw installation

OmegaClaw was installed locally under `projects/omegaclaw`. Important installation work:

- Cloned OmegaClaw-Core, PeTTa, nested OmegaClaw-Core runtime tree, and `petta_lib_chromadb`.
- Built local SWI-Prolog 9.3.36 from source because system SWI was absent/too old.

- Rebuilt SWI with required libraries: Janus, process, filesex, pcre, and uuid.
- Created a Python venv and installed OmegaClaw requirements.
- Downloaded/load-tested local embedding model `intfloat/e5-large-v2`.
- PeTTa smoke passed on `examples/fib.metta`.
- OmegaClaw mock startup/loop smoke passed.
- OpenClaw Gateway smoke returned an authenticated tiny response through the local `/v1/chat/completions` route.

6.2 OmegaClaw wrappers and supervisors

Local runtime wrappers and supervisors added:

```
projects/omegaclaw/local/run-omegaclaw-mock.sh
projects/omegaclaw/local/run-omegaclaw-openclaw-smoke.sh
projects/omegaclaw/local/omegaclaw-openclaw-supervisor.sh
projects/omegaclaw/local/run-omegaclaw-openclaw-telegram-private.sh
projects/omegaclaw/local/omegaclaw-telegram-private-supervisor.sh
projects/omegaclaw/local/check-omegaclaw-telegram-token.sh
projects/omegaclaw/local/telegram_openclaw_harness.py
bin/omegaclaw-watchdog.sh
```

The watchdog script checks the private Telegram supervisor PID, removes stale PID files, restarts through the supervisor, and reports `ALREADY_RUNNING`, `RESTARTED`, or `RESTART_FAILED`. A cron job wraps it and informs Ben only on restart/failure.

6.3 Telegram integration and safeguards

OmegaClaw/ProtoMegaBot Telegram functionality added after installation:

- Dedicated Telegram bot path for `@Protomegabot` / ProtomegaTron.
- Secret token stored only in local secret env file with restrictive permissions; token value omitted here.
- Telegram adapter supports `TG_ALLOWED_USER_ID(S)`, `TG_PRIVATE_ONLY`, and `TG_SKIP_INITIAL_OFFSET`.
- Landlock policy fixed to allow read-only access to `/run/systemd/resolve`, resolving Telegram polling DNS failures caused by `/etc/resolv.conf` symlink resolution.
- Natural-language OmegaClaw replies to fresh human messages are normalized/wrapped as Telegram `send` actions instead of being silently logged.
- Group routing adjusted when OpenClaw Telegram group policy had blocked Protobots messages.
- Privacy/routing debugging confirmed BotFather privacy was not the root blocker; OpenClaw group policy was.

6.4 Attachment handling

ProtoMegaBot originally could not see Telegram documents posted in the group. The runtime Telegram adapter was upgraded to:

- Download document attachments.
- Save attachments under `/home/openclaw/tmp/omegaclaw-telegram-attachments`.
- Inject text-like files into prompts as untrusted attachment content.
- Extract PDF text using local `pdftotext`.
- Enforce `TG_ATTACHMENT_MAX_BYTES` and `TG_ATTACHMENT_MAX_CHARS`.
- Chunk oversized extracted documents into `.chunkNNN.txt` files and give ProtoMegaBot first-chunk preview plus paths.

6.5 Loop, continuation, and acknowledgement behavior

After early no-op/anti-spam loops and deep-question stalls:

- A temporary one-inference-per-fresh-Telegram-input mitigation was replaced by a more explicit continuation protocol.
- Added `continue-thinking` skill/command, known-command normalization, `continueRequested` loop state, a `CONTINUATION-MSG` marker, and prompt instructions.
- Runner default `maxNewInputLoops` restored to 50 after avoiding idle OpenClaw calls.
- Prompt and adapter were tuned to provide brief acknowledgements for difficult/slow questions without boilerplate spam.
- For long/deep messages, an interface-level safeguard sends an immediate acknowledgement before handing the message to the LLM.
- Ben later requested that boilerplate acknowledgements be avoided; this preference is now durable memory and should be reflected in OmegaHive interaction policy.

6.6 Deep-call stall and backend diagnostics

Deep-call failures were traced to several issues:

- A fixed OpenClaw Gateway session for ProtoMegaBot had ballooned to roughly 998k/272k tokens.
- HTTP/subprocess timeouts of 180s/240s were too short for slow GPT-5.5 calls.
- Backend failures or empty output were converted into silent `()` instead of user-visible diagnostics.
- The Telegram supervisor could exit under `set -e` and leave a stale PID.

Fixes:

- Per-call Gateway sessions for OmegaClaw calls, since OmegaClaw already supplies its own prompt/history.
- 900s HTTP/subprocess timeout alignment.
- User-visible `(send ...)` diagnostics on backend failure/timeout/empty output.
- Supervisor changed so nonzero runner exits do not kill the supervisor permanently.

6.7 Context-overflow recovery and late-extension registry

On 2026-07-06, OmegaClaw-Core commit `98e8883` added:

- `lib_llm_ext.py` context-overflow recovery: error detection, preemptive escalation, GPT-5.5-class escalation knob, and chunk-summary fallback for oversized LLM responses.
- Telegram pending-message FIFO in `channels/telegram.py` to avoid burst messages being concatenated or lost through a single `_last_message` buffer.
- `lib_extensions.metta` / `src/extensions.py` status hook for `OMEGACLAW_LATE_EXTENSIONS`; `(extension-status)` reports loaded/failed/deferred/pending.
- Heavy deontic/directive imports kept deferred because in-loop `import!` can hang SWI even though top-level deontic tests pass. A true bounded/lazy loader or Prolog-backed API route is needed before enabling live in-process deontic/directive loading.

6.8 BotBotChats silence/routing fix

On 2026-07-07, a separate ProtoMegaBot BotBotChats silence bug was fixed in the nested OmegaClaw-Core runtime tree:

- Patched Telegram source-chat binding for dequeued inbound messages.
- Compacted duplicate `COMMAND_RETURN` feedback before storing `LAST_SKILL_USE_RESULTS`.
- Targeted stale skill-result replay, continuation loops, prompt ballooning, and wrong-channel substantive replies.
- Supervisor restarted with BotBotChats in target list.

7 ThreadKeeper hardening

ThreadKeeper is the OmegaClaw-side subagent/worker/delegation system. It has become a major source of OmegaHive-relevant design lessons.

Current relevant repository:

```
projects/omegaclaw/repos/ThreadKeeper
  branch agent/threadkeeper-hardening-next
  head 52a9bc9 Require string final emits
```

Important hardening changes include:

1. **Shell safety floor**: optional shell tool disabled by default, executable allowlist, argv-list execution with no `shell=True`, fixed workspace cwd, closed stdin, and fail-closed if workspace missing.

2. **Tool argument validation**: rejects null, empty, too-long, NUL-containing, wrong-type, and non-string tool arguments; rejects empty shell/search/tavily/technical-analysis queries.

3. **Output bounds**: caps read-file output, shell output, search/tavily/technical-analysis output, raw worker responses, native HTTP responses, final emits, transcript fields, transcript summaries, JSON audit/task reads, index audit reads, and transcript hash audit reads.

4. **Typed final emits**: final `emit` arguments must be strings; non-string emits become protocol violations rather than silently coerced strings.

5. **Queue and worker metadata**: worker results include per-task duration and total runtime; lock metadata includes tasks completed, consecutive errors, and remaining queue depth.

6. **Run-index bounding**: compact audit index can rotate to a configured number of retained entries while preserving a valid hash chain over retained entries.

7. **Audit bounds**: oversized `index.jsonl`, transcript JSON, and referenced transcript files are rejected or reported structurally before unbounded reads.

8. **Config validation**: non-finite numeric env values such as `nan` and `inf` are rejected and safe defaults are used.

9. **Runtime-tree sync**: hardened `src/subagent.py` is regularly copied/synced into the nested OmegaClaw-Core runtime tree and compile-checked.

Recent focused mock test counts rose from about 33 tests to 160 passing tests across the hardening sequence. No live Telegram/OmegaClaw runtime wiring was changed during these ThreadKeeper hardening slices unless explicitly noted; most were non-live code/test/doc changes.

OmegaHive recommendation: ThreadKeeper-like subagent infrastructure should be part of the core architecture, not an ad hoc add-on. It needs typed protocol boundaries, audit indices, redaction/adjudication, bounded persistence, queue contracts, and explicit operator-visible failure states.

8 GoalChainer, petta-memory, and non-live OmegaClaw gates

GoalChainer and petta-memory work has been used to prototype a safe decision/appraisal layer before live OmegaClaw/ZeroBot bridge changes.

Important additions:

- Installed/linked `omegaclaw-deontic` into the local OmegaClaw-Core tree so `lib_deontic` and `lib_directive` resolve.
- Added PeTTa library path symlink `PeTTa/OmegaClaw-Core -> repos/OmegaClaw-Core`.
- Fixed GoalChainer runtime seams so deontic/directive state surfaces ready/blocked/claim states.
- Demonstrated GoalChainer through Python CLI and through the actual OmegaClaw MeTTa skill surface.

- Added petta-memory evidence parsing and heuristic belief fusion for GoalChainer.
- Ran non-live smokes proving memory evidence can change belief strengths without overriding deontic verdicts.
- Built read-only sidecar and queue-sidecar contract artifacts for bounded appraisal of private ThreadKeeper task records.

Design conclusion: direct live bidirectional OmegaClaw–ZeroBot chat bridges are deferred. The preferred next bridge pattern is a supervised, queue-mediated, read-only sidecar where selected evidence and bounded task text are appraised, then only adjudicated/redacted summaries can egress.

9 Recovery and backup infrastructure

Private GitHub-backed recovery was added after Ben requested disaster recovery for both agents.

Recovery repositories:

```
bgoertzel-sing/zerobot-recovery       private
bgoertzel-sing/protomegabot-recovery  private
```

Recovery project files/scripts:

```
projects/agent-recovery/PROJECT.md
projects/agent-recovery/TASKS.md
projects/agent-recovery/DECISIONS.md
projects/agent-recovery/NOTES.md
projects/agent-recovery/scripts/backup-zerobot-recovery.sh
projects/agent-recovery/scripts/backup-protomegabot-recovery.sh
projects/agent-recovery/scripts/push-recovery-repos.sh
```

Current recovery state:

- Daily cron `Daily agent recovery GitHub backup` runs at 03:30 America/Vancouver.
- Backups exclude secrets, tokens, SSH keys, provider credentials, raw OpenClaw state, bulky repos, caches, venvs, and generated runtime artifacts.
- Secret scanning was tightened so internal hyphenated IDs no longer false-positive as OpenAI keys.
- Scripts fetch/rebase before push and verify repository cleanliness after backup.

OmegaHive recommendation: every hive agent should have a sanitized recovery bundle, restore documentation, secret-exclusion policy, scheduled backup, and periodic restore smoke test.

10 Channels and communication conventions

Current channel architecture:

- Direct Telegram with Ben for normal requests.

- Main Protobots group for concise important shared discussion.
- Scheduled-updates channel `telegram:-1003983157420` for scheduled/progress/alert traffic.
- ProtoBots-BotBotChats `telegram:-5459676079` for extended ZeroBot–ProtoMegaBot project discussion.

Durable routing conventions:

- Scheduled/progress updates go to scheduled-updates, except a single daily 7AM Pacific summary may stay in main Protobots.
- Bot-bot collective-loop discussions go to BotBotChats.
- Daily bot-bot project discussions are scheduled for active/idea projects and suppressed or disabled for paused/completed projects.
- Long bot-bot discussions should not dominate the main Protobots group.
- Avoid stock acknowledgements and boilerplate "I will work on it carefully" messages.

11 Known gaps and current risks

1. **Gateway restart pending**: Ollama provider config and `ollama/qwen2.5:7b` fallback need `sudo systemctl restart openclaw-agent.service` on the Pop!_OS laptop to become active in the running agent.
2. **Duplicate/legacy cron jobs**: there are both newer `daily bot-bot project discussion - ...` jobs and older `botbot daily discussion: ...` jobs for several projects. Scheduler reconciliation exists because this became confusing.
3. **Canonical remote ambiguity for OmegaClaw-Core runtime patches**: at least one runtime patch pushed only through a suspicious/misaligned fork remote; canonical upstream push failed with 403.
4. **Telegram token rotation**: earlier ProtoMegaBot token was pasted into chat and should be rotated if not already done. A debug env inspection on 2026-07-06 also exposed the token in tool output.
5. **Supervisor/watchdog is pragmatic**: ProtoMegaBot currently relies on cron watchdog restart. OmegaHive should have first-class service management, health probes, and structured restart policy.
6. **Deontic/directive live loading blocked**: in-loop SWI `import!` can hang; a bounded/lazy loader or Prolog-backed API route is needed before live deontic/directive imports.
7. **OpenRouter billing**: ThreadKeeper daily discussion cron hit OpenRouter 402 (insufficient credits) and OpenAI rate-limit cooldown on 2026-07-07. This is a billing issue, not a code bug.
8. **No active visible subagents at snapshot**: OpenClaw subagent visibility is tree-restricted; this inventory may miss sessions outside current visibility. Recovery repos and cron records are more reliable for durable state.

12 Source pointers

Primary source files and records used for this snapshot:

/home/openclaw/research-agent/catalog/INSTALLATION_KB.md
/home/openclaw/research-agent/catalog/install-kb/components.jsonl
/home/openclaw/research-agent/catalog/SETUP_REPORT.md
/home/openclaw/research-agent/MEMORY.md
/home/openclaw/research-agent/memory/2026-06-27.md
/home/openclaw/research-agent/memory/2026-06-28.md
/home/openclaw/research-agent/memory/2026-07-01.md
/home/openclaw/research-agent/memory/2026-07-02.md
/home/openclaw/research-agent/memory/2026-07-06.md
/home/openclaw/research-agent/memory/2026-07-07.md
/home/openclaw/research-agent/projects/agent-recovery/PROJECT.md
/home/openclaw/research-agent/projects/omegaclaw/PROJECT.md
/home/openclaw/research-agent/projects/omegaclaw/NOTES.md
/home/openclaw/research-agent/projects/openclaw-intent-model-router/PROJECT.md
/home/openclaw/research-agent/projects/channel-watchdog/PROJECT.md
/home/openclaw/research-agent/bin/omegaclaw-watchdog.sh
OpenClaw cron registry as listed on 2026-07-07 15:56 Pacific
OpenClaw session_status as observed on 2026-07-07
Skill Workshop proposal list as observed on 2026-07-07